PROMPT ENGINEERING MASTERY BUCKET · MODULE 13

AI Workflows

Bringing AI into daily engineering and business workflows — VS Code + AI, Claude Code, GitHub Copilot, Cursor, Windsurf, and automation platforms n8n, Make, and Zapier.

DIFFICULTY LEVEL
Beginner–Intermediate

ESTIMATED READING TIME
50–65 minutes

ESTIMATED PRACTICE TIME
80–100 minutes

PREREQUISITES
Modules 01–12

LEARNING OUTCOMES

By the end of this module, you will be able to: distinguish AI-powered coding tools by their working style; choose the right coding assistant for a given workflow; understand no-code/low-code automation platforms and where AI fits into them; and combine coding and automation tools into a coherent personal or team AI workflow.

Table of Contents

1. Introduction

What is this topic?

Every module so far has focused on building AI features from the API up. This module zooms out to the tools that bring AI directly into existing daily workflows — coding editors with AI built in, and no-code automation platforms that let AI trigger and participate in business processes without custom code.

Why is it important?

Not every AI use case needs a custom-built application. Often the highest-leverage move is bringing AI into a workflow you already have — your editor, your automation pipeline — rather than building something from scratch.

Why should AI Engineers learn it?

These tools are how most of this course's concepts (agents, Module 08; tool use, Module 06) reach everyday work in practice — a coding agent in your editor is Module 08's agent loop, productized. Knowing the landscape helps you pick the right tool instead of defaulting to whatever's trending.

Where is it used in the real world?

Developers use AI-native editors and coding assistants daily for writing and reviewing code; operations and business teams use no-code automation platforms with AI steps to handle repetitive multi-app workflows without engineering involvement.

Why is it relevant today?

AI-assisted coding tools and AI-augmented automation platforms have become mainstream parts of both software development and general business operations in 2025-2026 — familiarity with this landscape is now broadly useful, not niche.

> ⚠ **Warning**
> Specific features and capabilities of each tool below evolve quickly. This module covers each tool's general working philosophy and best-fit use cases — check each product's current documentation for exact, up-to-date feature details.

2. Before You Begin

RECAP

Agent loops (Module 08, 3.6) — many of the coding tools below are, under the hood, running an agent loop against your codebase.

RECAP

Tool use (Module 06) — automation platforms like n8n, Make, and Zapier are essentially visual, no-code interfaces for building tool-using AI workflows.

Tip

Try to categorize each new AI tool you encounter by "working style" (Section 4) rather than by name alone — it transfers your understanding to new tools that come out after this module was written.

3. Main Lesson

Each tool follows: **Definition → Simple Explanation → Analogy → Strengths → Considerations → Best Use Cases → Common Mistakes → Summary.**

💻3.1 VS Code + AI

DEFINITION

VS Code + AI refers to using AI extensions/integrations (including Copilot, Claude, and others) within Microsoft's popular, free, extensible code editor, rather than switching to a dedicated AI-native editor.

SIMPLE EXPLANATION

Since VS Code has a huge extension ecosystem, AI capability is typically added as an extension on top of an editor you likely already know, rather than requiring you to switch your whole development environment.

> 💡 *Like adding a powerful new app to a phone you already own and know how to use, instead of buying an entirely new phone designed around that one app.*

STRENGTHS

Familiar, widely-used base editor; huge non-AI extension ecosystem still available alongside AI features; free and highly customizable.

CONSIDERATIONS

AI integration depth varies by extension — some are lighter-weight autocomplete-style additions, others (like a Claude Code extension) offer deeper agentic capability.

BEST USE CASES

Developers who want AI assistance without leaving their existing, familiar editor and its broader extension ecosystem.

COMMON MISTAKES

Assuming all AI extensions in VS Code offer the same depth of capability — some are simple autocomplete, others are full agentic coding tools.

SUMMARY

VS Code + AI brings AI capability into an already-familiar, extensible editor — a low-friction way to add AI assistance to an existing workflow.

🤖3.2 Claude Code

DEFINITION

Claude Code is Anthropic's agentic coding tool, letting developers delegate coding tasks to Claude directly from the command line, a desktop app, or as an integration within editors — running an agent loop (Module 08) against a real codebase.

SIMPLE EXPLANATION

Rather than just suggesting the next line as you type, Claude Code can autonomously read a codebase, plan changes across multiple files, write and run code, and iterate based on test results — the full agent loop pattern from Module 08 applied to real software engineering tasks.

> 💡 *Like handing a well-scoped ticket to a capable junior engineer and letting them work through it end to end — reading the code, making changes, running tests — rather than pairing on every single keystroke.*

STRENGTHS

Deep, autonomous multi-file/multi-step task handling; strong fit for Claude's noted coding

strengths (Module 11, 3.2); flexible access (terminal, desktop app, editor integration).

CONSIDERATIONS

Autonomous multi-step changes benefit from the same safeguards covered in Module 08 (bounded steps) and Module 10 (guardrails, confirmation for consequential actions) — review agentic changes before merging, especially early on.

BEST USE CASES

Well-scoped coding tasks (bug fixes, feature implementation, refactors, test writing) that benefit from autonomous multi-step execution rather than line-by-line assistance.

COMMON MISTAKES

Giving vague, underspecified tasks and expecting perfect autonomous results — like any agent (Module 08, 3.2), clear goals and scope improve outcomes significantly.

SUMMARY

Claude Code applies the agent loop pattern directly to software engineering — autonomous, multi-step task execution rather than simple autocomplete.

🤖 3.3 GitHub Copilot

DEFINITION

GitHub Copilot is GitHub/Microsoft's AI coding assistant, one of the earliest and most widely-adopted AI pair-programming tools, integrated into popular editors including VS Code.

SIMPLE EXPLANATION

Copilot's original core experience centered on inline code suggestions as you type, and has expanded over time toward broader chat-based and more agentic capabilities within its ecosystem.

> 💡 *Like a highly experienced pair-programming partner sitting beside you, suggesting the next few lines as you type — originally a tight, in-the-moment collaboration style, expanding toward bigger-picture assistance over time.*

STRENGTHS

Extremely wide adoption and editor integration, tight GitHub ecosystem integration (PRs, issues), mature inline-suggestion experience.

CONSIDERATIONS

Feature depth (inline suggestions vs. deeper agentic capability) varies by product tier and has evolved substantially over time — check current documentation for the latest capability set.

BEST USE CASES

Developers wanting tightly-integrated, in-the-moment coding assistance within a GitHub-centric workflow.

COMMON MISTAKES

Assuming Copilot's capabilities are frozen at "inline autocomplete only" without checking its current, evolving feature set.

SUMMARY

GitHub Copilot is a widely-adopted, deeply GitHub-integrated coding assistant, historically centered on inline suggestions with expanding capabilities over time.

🖱 3.4 Cursor

DEFINITION

Cursor is an AI-native code editor — built from the ground up around AI-assisted coding, rather than AI added as an extension to an existing editor.

SIMPLE EXPLANATION

Because Cursor is built AI-first (as a fork of VS Code's foundation, but designed specifically around AI workflows), its AI features tend to be more deeply woven into the core editing experience than editor-plus-extension approaches.

> 💡 *Like a car designed from scratch around electric propulsion, versus a traditional car retrofitted with an electric motor — both can work well, but a ground-up design can integrate the new capability more deeply.*

STRENGTHS

Deep, native AI integration throughout the editing experience; often quick to adopt new AI coding capabilities given its AI-first focus.

CONSIDERATIONS

Requires switching editors if you're not already using it (unlike the VS Code + extension approach); smaller non-AI extension ecosystem than VS Code itself.

BEST USE CASES

Developers wanting the deepest possible native AI-coding experience and comfortable switching their primary editor to get it.

COMMON MISTAKES

Switching to an AI-native editor purely for hype without evaluating whether the switching cost is worth it for your specific workflow needs.

SUMMARY

Cursor is an AI-native editor with deep, built-in AI integration — a stronger commitment than adding AI to an existing editor, with a corresponding switching cost.

🏄3.5 Windsurf

DEFINITION

Windsurf is another AI-native code editor, positioned similarly to Cursor as a ground-up AI-first coding environment, with its own distinct approach to agentic coding workflows within the editor.

SIMPLE EXPLANATION

Like Cursor, Windsurf builds AI deeply into the core editor experience rather than as an add-on — the specific competitive differentiators between AI-native editors like Windsurf and Cursor shift frequently as both evolve rapidly.

> 💡 *Like two competing ground-up electric vehicle designs — both start from the same "AI-native" premise, but differ in specific execution and design choices worth comparing directly for your needs.*

STRENGTHS

Deep native AI integration (shared with Cursor's category); active, fast-evolving feature development typical of AI-native editor competition.

CONSIDERATIONS

As with Cursor, requires an editor switch; specific differentiators vs. competitors change quickly given the pace of development in this space.

BEST USE CASES

Developers evaluating AI-native editors who want to compare Windsurf's specific approach directly against alternatives like Cursor for their workflow.

COMMON MISTAKES

Picking between AI-native editors based on general reputation alone rather than trying

both directly on your own real tasks.

SUMMARY

Windsurf is an AI-native editor in the same category as Cursor — direct hands-on comparison is the most reliable way to choose between them for your workflow.

🔗3.6 n8n

DEFINITION

n8n is an open-source, self-hostable workflow automation platform that lets users visually build multi-step automations connecting different apps and services, including AI/LLM steps.

SIMPLE EXPLANATION

Instead of writing custom code to connect a form submission to an LLM call to a database write to a Slack notification, you build that chain visually as connected nodes — with an "AI Agent" node type available for LLM-powered steps, essentially agent loops (Module 08) built through a visual interface.

> 💡 *Like a visual flowchart you can actually run — each box is a real action (send email, call an API, ask an LLM), and connecting boxes together builds a working automation without writing code.*

STRENGTHS

Open-source and self-hostable (data control), visual workflow building accessible to non-engineers, flexible custom node/code steps when needed.

CONSIDERATIONS

Self-hosting requires some infrastructure setup/maintenance (a managed cloud option is also typically available); visual workflows can become complex to debug at scale.

BEST USE CASES

Teams wanting open-source/self-hosted automation with data control, and workflows mixing AI steps with traditional app integrations (forms, databases, notifications).

COMMON MISTAKES

Building extremely complex visual workflows that become hard to debug — sometimes a small amount of custom code within a node is clearer than an overly branched visual flow.

SUMMARY

n8n is an open-source visual automation platform with AI-step support — strong for teams wanting self-hosted control over their automation infrastructure.

🧩3.7 Make

DEFINITION

Make (formerly Integromat) is a cloud-based, visual workflow automation platform connecting apps and services, including AI/LLM integrations, with a particularly visual, flowchart-like builder interface.

SIMPLE EXPLANATION

Similar in purpose to n8n, but cloud-only (no self-hosting option) with a strong emphasis on a highly visual, branching-flow interface for building automations, including steps that call LLMs for text generation, classification, or decision-making within a flow.

> 💡 *Like a well-organized subway map for your business processes — visually tracing exactly how data and actions flow from one step to another, with clear branch points for different conditions.*

STRENGTHS

Polished, highly visual builder interface, large library of pre-built app integrations, no infrastructure to manage (fully cloud-hosted).

CONSIDERATIONS

Cloud-only — no self-hosting option for teams needing that data control; pricing scales with usage/complexity as with most cloud automation platforms.

BEST USE CASES

Teams wanting a polished, easy-to-learn visual automation tool without any infrastructure management, especially for business/ops-driven (not engineering-driven) automation.

COMMON MISTAKES

Choosing Make when self-hosting/data control is actually a hard requirement — that scenario fits n8n's model better.

SUMMARY

Make offers a polished, fully-managed visual automation experience — strong for ease of use, without a self-hosting option.

⚡3.8 Zapier

DEFINITION

Zapier is one of the earliest and most widely-adopted cloud automation platforms, connecting apps via simple trigger-action "Zaps," with AI steps available for LLM-powered actions within a Zap.

SIMPLE EXPLANATION

Zapier's core mental model is simpler than n8n or Make's branching flowcharts: a trigger ("when this happens") followed by one or more actions ("do this") — including an AI action step that can generate text, summarize, or classify as part of the chain.

> 💡 *Like a simple "if this, then that" rule you set once and forget — the most beginner-friendly entry point into automation, trading some flexibility for extreme simplicity.*

STRENGTHS

Very large app integration library, simplest mental model for beginners (trigger → action), strong brand recognition and community resources.

CONSIDERATIONS

Historically less suited to complex, highly-branching logic compared to Make's visual flow builder, though its capabilities have grown over time — check current features for complex use cases.

BEST USE CASES

Simple, straightforward automations (trigger → AI action → notify) especially for beginners or teams wanting the fastest path to a working automation.

COMMON MISTAKES

Forcing a genuinely complex, highly-branched workflow into Zapier's simpler trigger-action model when a more flexible tool (Make, n8n) would fit better.

SUMMARY

Zapier offers the simplest, most beginner-friendly automation model — ideal for straightforward trigger-action workflows including AI steps.

4. Visual Learning

🌳 Coding Tools by Working Style

```
AI Coding Tools
├── Extension on existing editor (VS Code + AI, incl. Copilot, Claude Code extension)
└── AI-native editor (Cursor, Windsurf)
Standalone Agentic CLI/App
└── Claude Code (terminal / desktop / editor integration)
```

🌳 Automation Platforms by Model

```
Automation Platforms
├── Self-hostable, open-source (n8n)
├── Cloud, visual flowchart builder (Make)
└── Cloud, simple trigger→action (Zapier)
```

📊 Coding Tool Comparison

Tool	Working Style	Best For
VS Code + AI	Extension on familiar editor	Low-friction AI addition
Claude Code	Autonomous agent (Module 08)	End-to-end delegated coding tasks
GitHub Copilot	Inline suggestions, GitHub-integrated	In-the-moment pair programming
Cursor	AI-native editor	Deep, built-in AI coding experience
Windsurf	AI-native editor	Same category as Cursor, compare directly

5. Real World Applications

Scenario	Common Tool Fit
Delegating a well-scoped feature/bug fix end to end	Claude Code (autonomous agent loop)
Wanting AI help without leaving a familiar editor	VS Code + AI extensions
Business team automating "new form submission → AI summary → Slack message"	Zapier (simple) or Make (more complex branching)
Team needing self-hosted automation for data control	n8n

6. Practical Examples

Beginner: Categorizing tools

Sort all 8 tools from Section 3 into the two trees from Section 4 (coding tools by working style, automation platforms by model) from memory, then check your answers.

Beginner: Matching tool to task

For each: (1) a solo developer wanting quick AI autocomplete in their existing editor, (2) a non-technical ops person automating a simple Slack notification, (3) a team needing self-hosted automation infrastructure — name the tool you'd suggest.

Intermediate: Designing a simple automation

Sketch (on paper) a Zapier-style trigger → AI action → notify workflow for a real task you have (e.g., summarizing new emails into a daily digest).

Advanced: Delegating a real coding task

Write a clear, well-scoped task description (following Module 08's planning principles) that you could hand to an agentic coding tool like Claude Code, and identify what could go wrong if the description were vague instead.

7. Code Examples

🔧Conceptual — An n8n-style AI automation node (pseudocode)

```
// Conceptual representation of a visual n8n/Make workflow, in pseudocode
// A real n8n workflow is built visually, not by writing this directly

Trigger: "New row added to Google Sheet (customer feedback)"
  |
  ▼
AI Node: Summarize + classify sentiment
  input: {{ row.feedback_text }}
  prompt: "Classify sentiment (Positive/Negative/Neutral) and
           summarize in 1 sentence: {{ row.feedback_text }}"
  model: claude-sonnet-4-6
  |
  ▼
Condition: If sentiment == "Negative"
  |
  ▼ (yes)
Action: Send Slack message to #customer-issues
  message: "⚠ Negative feedback: {{ ai_summary }}"
```

🐍Python — The "custom code" equivalent of that automation

```
# pip install anthropic
# This is what the visual n8n/Make workflow above is doing under the hood
import anthropic

client = anthropic.Anthropic(api_key="YOUR_API_KEY")

def process_feedback_row(feedback_text):
    prompt = f"""Classify sentiment (Positive/Negative/Neutral) and
summarize in 1 sentence: {feedback_text}"""

    response = client.messages.create(
        model="claude-sonnet-4-6", max_tokens=100,
        messages=[{"role": "user", "content": prompt}]
    )
    result = response.content[0].text

    if "Negative" in result:
        send_slack_alert(f"⚠ Negative feedback: {result}")   # your own function

    return result
```

> **Tip**
> Comparing the pseudocode workflow to the equivalent Python makes the value proposition of no-code platforms concrete: they trade some flexibility for a much lower barrier to building the same underlying logic.

8. Common Mistakes

- **Assuming all AI coding extensions offer equal depth** — capabilities range from simple autocomplete to full agentic task execution.
- **Giving agentic coding tools vague, underspecified tasks**, then blaming the tool for a poor result.
- **Switching to an AI-native editor purely for hype** without evaluating real switching costs and benefits.
- **Building overly complex visual automation workflows** that become hard to debug.
- **Choosing a cloud-only platform (Make/Zapier)** when self-hosting/data control is actually a hard requirement.
- **Forcing complex branching logic into Zapier's simpler trigger-action model** when a more flexible tool would fit better.

9. Best Practices

- **Categorize new AI tools by working style** (extension vs. native vs. standalone agent) to transfer understanding to new tools quickly.
- **Give agentic coding tools clear, well-scoped tasks**, applying Module 08's planning principles.
- **Review agentic code changes before merging**, especially for consequential or unfamiliar codebases.
- **Match automation platform to actual requirements**: self-hosting need → n8n; visual complexity → Make; simplicity → Zapier.
- **Trial AI-native editors hands-on** on real tasks before committing to a switch.

10. Hands-on Activities

Easy 5 exercises

1. List all 8 tools from Section 3 and one distinguishing feature for each, from memory.

2. Explain the difference between an "AI extension on an existing editor" and an "AI-native editor" in your own words.

3. For a non-technical ops team member, which automation platform would you recommend first, and why?

4. Sketch a simple trigger → AI action → notify automation for a task in your own life or work.

5. Explain why self-hosting (n8n) might matter more to some teams than others.

Intermediate 3 exercises

1. If you have access to any AI coding tool (VS Code extension, Claude Code, Cursor, etc.), give it a well-scoped small task and observe how it approaches it.

2. Build (or sketch in detail) an automation with a branching condition (like the Section 7 pseudocode) for a real use case.

3. Compare the pseudocode automation from Section 7 against writing the equivalent as custom code — list 2 advantages of each approach.

Challenge 2 exercises

1. Design a combined workflow using both a coding tool (for building something) and an automation platform (for running it repeatedly) — describe how they'd connect.

2. Research current feature sets for 2 AI-native editors (e.g., Cursor and Windsurf) and write a short comparison of their current differentiators.

11. Mini Project

Goal

Design and (if platform access is available) build a simple AI-powered automation: new text input → AI classification/summary → conditional notification — first in a no-code platform, then as equivalent custom code.

Requirements

- Access to at least one automation platform (n8n, Make, or Zapier — free tiers are typically available) OR willingness to design it on paper
- An Anthropic API key for the custom-code version

Step-by-Step Instructions

1. Pick a real, simple use case (e.g., classify incoming feedback, summarize a form submission).
2. Design (or build) the workflow visually: trigger → AI step → condition → action.
3. Separately, write the equivalent as custom Python/JavaScript code calling the Anthropic API directly.
4. Compare: which was faster to build? Which is easier to modify later? Which fits your actual skill set and needs better?

Expected Output

```
No-code version: [screenshot or diagram of your visual workflow]
Custom code version: [your Python/JS script]

Comparison:
- Build time: no-code faster / custom code faster (circle one, with reasoning)
- Flexibility: [notes on which handled edge cases better]
- Maintenance: [notes on which was easier to modify/debug]
```

Possible Improvements

- Add error handling to the custom-code version and see how much extra effort it takes compared to the no-code platform's built-in handling.
- Test the same automation with a genuinely complex branching case and note where each approach starts to strain.
- Try delegating the custom-code version's implementation itself to an agentic coding tool like Claude Code.

12. Review Questions

Multiple Choice (10)

1. Claude Code is best described as:

A) A simple autocomplete plugin B) An agentic coding tool running an agent loop against a codebase C) A no-code automation platform D) A model comparison tool

2. Cursor and Windsurf share which characteristic?

A) Both are cloud automation platforms B) Both are AI-native code editors C) Both require no editor at all D) Both are Anthropic products

3. GitHub Copilot's original core experience centered on:

A) Visual workflow automation B) Inline code suggestions as you type C) No-code app building D) Model training

4. n8n's key differentiator among automation platforms is:

A) Being closed-source only B) Being open-source and self-hostable C) Having no AI support D) Requiring no visual interface

5. Make is best known for:

A) A polished, visual flowchart-style workflow builder B) Being a coding editor C) Self-hosting only D) Having no app integrations

6. Zapier's core mental model is:

A) Complex branching flowcharts only B) Simple trigger → action C) A coding editor D) Model fine-tuning

7. VS Code + AI approaches typically involve:

A) Building a new editor from scratch B) Adding AI capability via extensions to an existing, familiar editor C) No AI capability at all D) Only visual automation

8. Why should agentic coding tools be given clear, well-scoped tasks?

A) It's not necessary B) Clear scope significantly improves autonomous task outcomes C) It reduces token count to zero D) It's required by law

9. A team requiring self-hosted automation for data control would most likely choose:

A) Zapier B) Make C) n8n D) Cursor

10. AI-native editors like Cursor and Windsurf differ from VS Code + AI extensions by:

A) Having no AI features B) Building AI deeply into the core editor from the ground up C) Being automation platforms D) Requiring no installation

Identification (5)

1. _____ is Anthropic's agentic coding tool, delegating coding tasks via an autonomous agent loop.

2. _____ is an AI-native code editor built from the ground up around AI-assisted coding.

3. _____ is an open-source, self-hostable visual workflow automation platform.

4. _____ is a cloud automation platform known for a polished, visual flowchart-style builder.

5. _____ is a cloud automation platform built around a simple trigger → action model.

True or False (5)

1. All AI coding extensions offer the exact same depth of capability.

2. Claude Code can autonomously read a codebase, make multi-file changes, and iterate based on test results.

3. n8n offers a self-hosting option, unlike Make or Zapier.

4. Zapier is generally the most beginner-friendly automation platform of the three covered.

5. Reviewing agentic coding tool output before merging is a reasonable safety practice.

Situational (5)

1. You want deep AI coding help but don't want to change your editor. What category of tool fits best?

2. You want to delegate a well-scoped bug fix end to end and review the result afterward. What tool fits this?

3. A non-technical teammate wants to automate "new lead → AI-drafted welcome email → CRM update" as simply as possible. What platform would you suggest first?

4. Your company has strict data residency rules affecting your automation infrastructure choice. What platform category should you look at?

5. You're evaluating whether to switch your primary editor to an AI-native one. What's the recommended way to decide?

Answer Key

Multiple Choice: 1-B, 2-B, 3-B, 4-B, 5-A, 6-B, 7-B, 8-B, 9-C, 10-B

Identification: 1-Claude Code, 2-Cursor (or Windsurf), 3-n8n, 4-Make, 5-Zapier

True or False: 1-False, 2-True, 3-True, 4-True, 5-True

Situational (sample reasoning):

1. An AI extension on your existing editor (VS Code + AI).
2. An agentic coding tool like Claude Code.
3. Zapier — simplest trigger → action model, most beginner-friendly.
4. Self-hostable platforms like n8n.
5. Trial it hands-on with real tasks rather than deciding from reputation alone.

13. Cheat Sheet

💻 Coding Tools

VS Code + AI — extension, familiar editor.
Claude Code — autonomous agent.
Copilot — inline suggestions.
Cursor/Windsurf — AI-native editors.

🔗🐙⚡ Automation

n8n — self-hosted, open-source.
Make — cloud, visual flowchart.
Zapier — cloud, simple trigger→action.

🎯 Choosing

Categorize by working style, then match to your actual requirement (data control, complexity, ease of use).

✔ Agentic Tasks

Clear scope in → better autonomous results out. Review before merging.

14. Key Takeaways

- AI coding tools range from lightweight extensions on familiar editors to fully AI-native editors to standalone autonomous agents like Claude Code.
- Claude Code applies the agent loop pattern (Module 08) directly to real software engineering tasks.
- AI-native editors (Cursor, Windsurf) integrate AI more deeply than extension-based approaches, at the cost of switching your primary editor.
- No-code automation platforms (n8n, Make, Zapier) let AI participate in business workflows visually, without custom code.
- n8n offers self-hosting; Make offers a polished visual builder; Zapier offers the simplest trigger-action model — choose based on actual requirements.
- Clear, well-scoped tasks significantly improve outcomes from any agentic coding tool, consistent with Module 08's planning principles.

15. Additional Resources

Official Documentation

- Anthropic — Claude Code documentation (docs.claude.com)
- GitHub Copilot documentation
- Cursor and Windsurf official documentation
- n8n, Make, and Zapier official documentation

Related Modules

- Module 08 — AI Agents (the agent loop pattern underlying agentic coding tools)
- Module 06 — Function Calling / Tool Use (the mechanism automation platforms build on visually)

Communities & Further Learning

- Each tool's official changelog/release notes — this category evolves especially quickly.